Nandana Rajesh

Roll No: 44

GUIDE: Mr.Sivadas T Nair

Problem Statement

Sl. No	Citation	Methodology
1	IMD Weather Monitoring System	Collects real-time meteorological data using sensors and satellites
2	NDMA Disaster Alert System	Issues alerts based on predefined disaster thresholds
3	Weather Apps (e.g., AccuWeather)	Provides real-time forecasts using API-based data aggregation
4	Research on Climate Risk Models	Uses statistical and threshold-based models for risk classification
5	Flood Prediction Systems	Uses rainfall and water-level data for predictive analysis

EXISTING SYSTEMS

WEATHER APPLICATIONS PROVIDE TEMPERATURE, HUMIDITY, AND RAINFALL DATA

DISASTER MANAGEMENT SYSTEMS ISSUE ALERTS DURING EXTREME CONDITIONS

BASIC FORECASTING SYSTEMS PREDICT SHORT-TERM WEATHER PATTERNS

Problems Identified

- No **integrated risk classification system**
- Lack of **localized analysis (city-specific like Kochi)**
- Alerts are **not actionable or user-friendly**
- No connection between **weather data and emergency preparedness**
- Systems are **reactive rather than proactive**

Proposed System

To convert raw weather data into **meaningful risk levels**

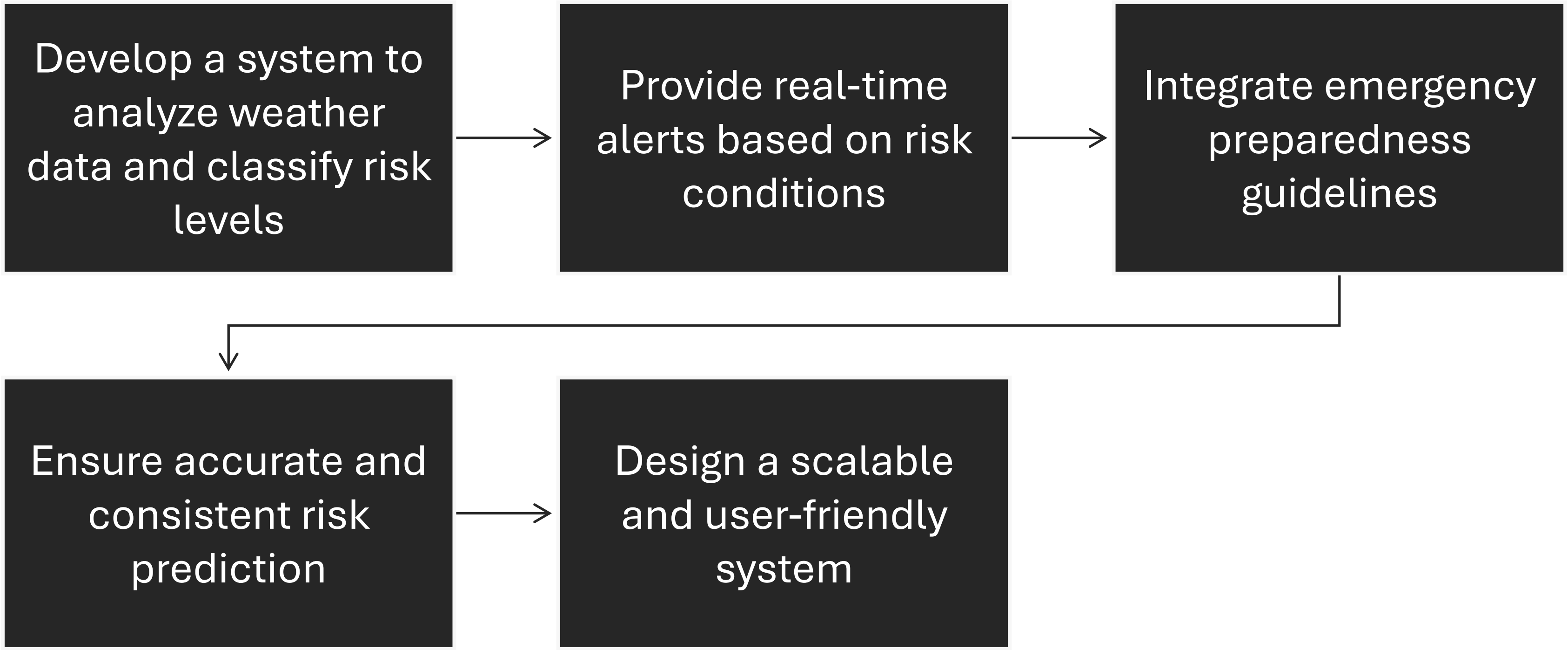
To provide **early warnings with actionable insights**

To integrate **risk analysis + alerts + preparedness**

To improve **public awareness and safety**

To support **decision-making during extreme weather events**

Project Objectives



Scope

Covers weather-based risk analysis for a specific region (e.g., Kochi)

Includes alert generation and emergency recommendations

Supports both simulated and real API-based data integration

Designed for future expansion to multiple regions

Expected Users

- General public
- Local residents in risk-prone areas
- Disaster management authorities
- Researchers and analysts

Requirement Analysis

Functional Requirements

- User registration and authentication
- Fetch and store weather data
- Perform risk analysis using multiple parameters
- Classify risk levels (Low / Medium / High)
- Generate and display alerts
- Provide emergency preparedness recommendations
- Store and manage historical data

Non-Functional Requirements

- **Performance:** System should respond within 2 seconds
- **Scalability:** Should handle increasing user load
- **Reliability:** Accurate and consistent risk predictions
- **Security:** Protect user data and system access
- **Usability:** Simple and intuitive interface
- **Maintainability:** Easy to update thresholds and logic

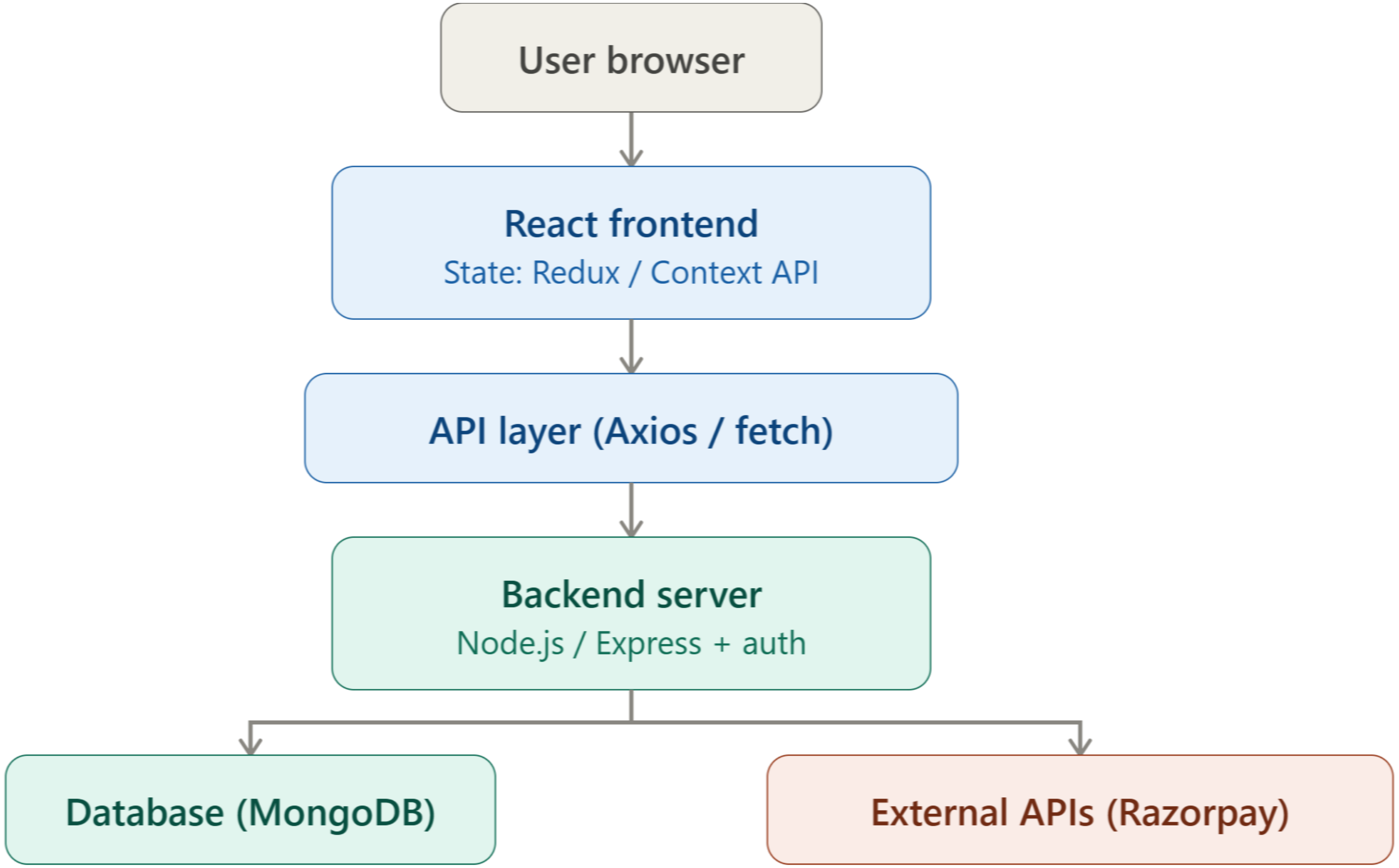
Overall Architecture

Weather Guard follows a 3-tier client-server architecture:

- Presentation Layer (Frontend) – React.js
  Handles UI rendering and user interaction, and communicates with the backend over HTTP/HTTPS. It does not call external APIs directly; all requests go through the backend.
- Application Layer (Backend) – Spring Boot
  Exposes REST APIs and contains all business logic, including authentication, risk computation, and alert handling. This is the only layer that communicates with external services and the database.
- Data Layer – MySQL
  Stores users, weather records, computed risk data, alerts, and emergency guidelines.

External Dependency: Open-Meteo weather API, accessed only by the backend, never exposed to the frontend directly.

Architecture Diagram



Modules

- User Module – handles registration, login, JWT-based authentication, and profile/location preferences.
- Weather Data Module – fetches live weather data from Open-Meteo and formats it for the frontend.
- Risk Analysis Module – uses the RiskAnalysisService to process weather data, apply seasonal baselines and threshold/compound rules, and output a risk level (Low, Medium, or High) with a reason.
- Alerts Module – generates and categorizes alerts by severity, using a hybrid mechanism of real API data where available and admin-managed alerts otherwise.
- Emergency Preparedness Module – serves safety guidelines matched to the current risk level and disaster type.
- Data Visualization Module – formats historical and trend weather data for charts on the frontend.
- Admin Module – allows admins to manage alerts and monitor system activity, restricted to users with an admin role.

Tables

User

Field Name	Data Type / Constraint	Description
user_id	INT, Primary Key, Auto Increment	Unique identifier for each user
username	VARCHAR, UNIQUE, NOT NULL	Username of the user
email	VARCHAR, UNIQUE, NOT NULL	Email address of the user
password_hash	VARCHAR, NOT NULL	Encrypted password for authentication
role	ENUM ('user', 'admin')	Defines user role (normal user or admin)
location_id	INT, Foreign Key	References Location(location_id)
created_at	TIMESTAMP	Timestamp of account creation

Location

Field Name	Data Type / Constraint	Description
location_id	INT, Primary Key, Auto Increment	Unique identifier for location
city	VARCHAR	Name of the city
state	VARCHAR	Name of the state
latitude	DECIMAL	Latitude coordinate
longitude	DECIMAL	Longitude coordinate

Weather data

Field Name	Data Type / Constraint	Description
weather_id	INT, Primary Key, Auto Increment	Unique identifier for weather record
location_id	INT, Foreign Key	References Location(location_id)
temperature	DECIMAL	Temperature value
humidity	DECIMAL	Humidity level
rainfall	DECIMAL	Rainfall measurement
wind_speed	DECIMAL	Wind speed value
timestamp	TIMESTAMP	Time of data recording

Risk data

Field Name	Data Type / Constraint	Description
risk_id	INT, Primary Key, Auto Increment	Unique identifier for risk record
weather_id	INT, Foreign Key	References WeatherData(weather_id)
risk_level	ENUM ('Low', 'Medium', 'High')	Classified risk level
temperature	DECIMAL	Temperature used for risk calculation
rainfall	DECIMAL	Rainfall used for risk calculation
wind_speed	DECIMAL	Wind speed used for risk calculation
humidity	DECIMAL	Humidity used for risk calculation
description	VARCHAR	Explanation of risk level
timestamp	TIMESTAMP	Time of risk evaluation

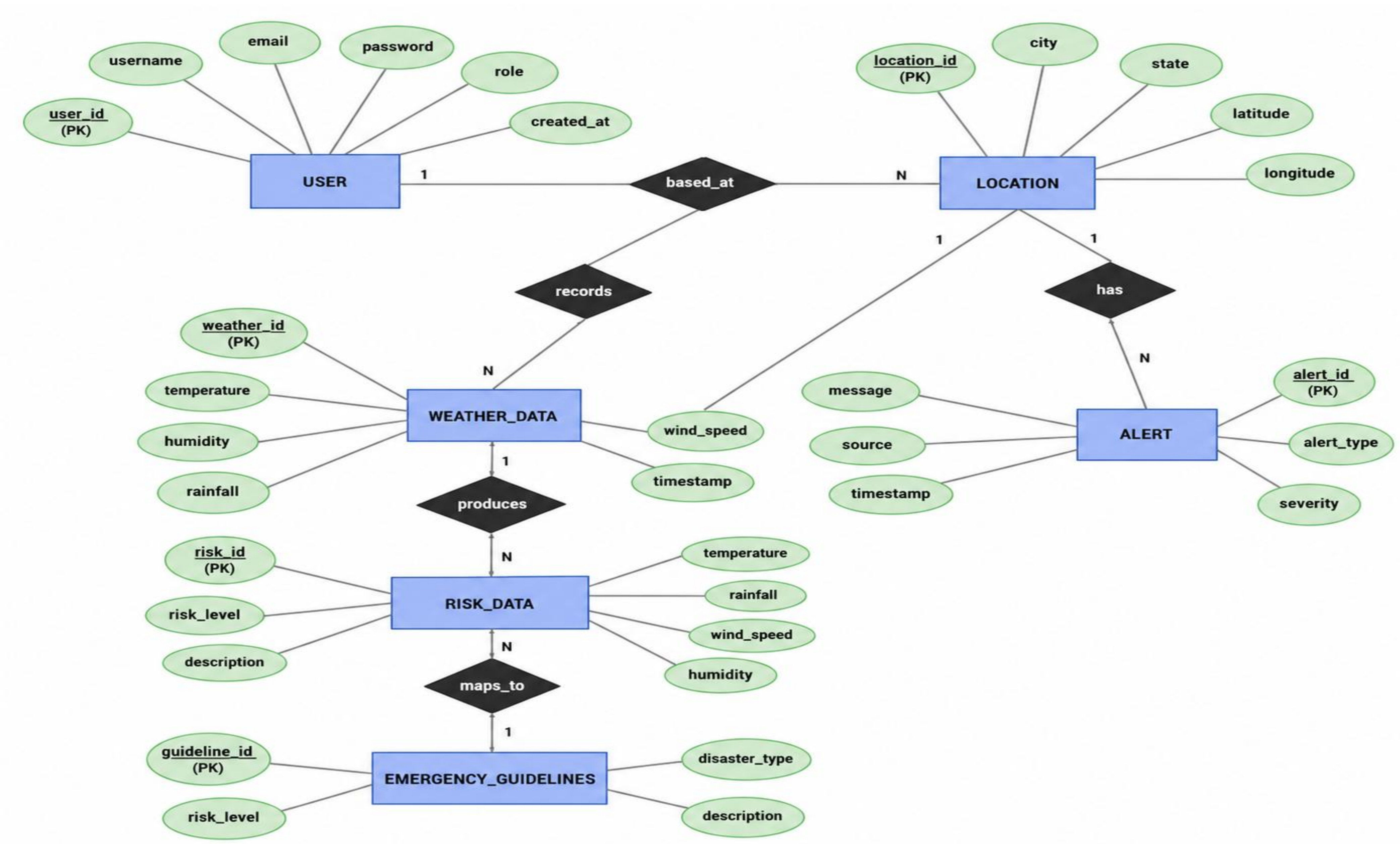
Alert

Field Name	Data Type / Constraint	Description
alert_id	INT, Primary Key, Auto Increment	Unique identifier for alert
location_id	INT, Foreign Key	References Location(location_id)
alert_type	VARCHAR	Type of alert (e.g., Rain, Heatwave, Storm)
severity	ENUM ('Low', 'Medium', 'High')	Severity level of alert
message	VARCHAR	Alert message content
source	ENUM ('API', 'Admin')	Source of alert (external API or admin)

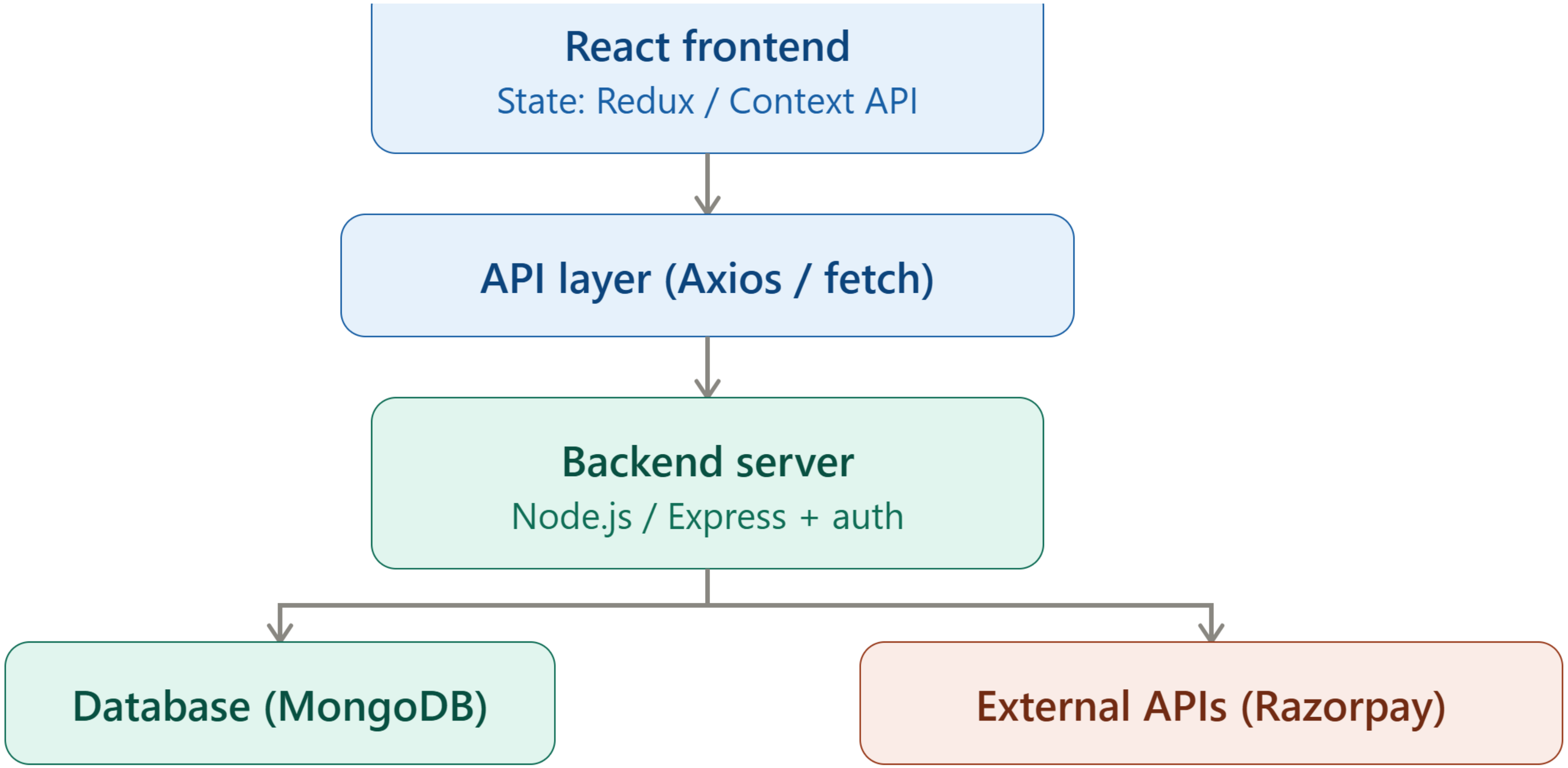
Emergency Guidelines

Field Name	Data Type / Constraint	Description
guideline_id	INT, Primary Key, Auto Increment	Unique identifier for each guideline
risk_level	ENUM ('Low', 'Medium', 'High')	Associated risk level
disaster_type	VARCHAR	Type of disaster (e.g., Flood, Heatwave, Storm)
description	TEXT	Detailed emergency instructions
timestamp	TIMESTAMP	Time of guideline creation/update

ER Diagram


username
email
password
role
user_id
(PK)
created_at
USER
1
based_at
N
LOCATION
location_id
(PK)
city
state
latitude
longitude
1
1
records
has
weather_id
(PK)
N
N
temperature
WEATHER_DATA
humidity
rainfall
wind_speed
timestamp
1
produces
message
source
timestamp
ALERT
alert_id
(PK)
alert_type
severity
risk_id
(PK)
N
risk_level
description
RISK_DATA
temperature
rainfall
wind_speed
humidity
N
maps_to
1
guideline_id
(PK)
risk_level
EMERGENCY_GUIDELINES
disaster_type
description

Technology Stack



Product Backlog

ID	User Story
US1	As a user, I want to view real-time weather updates so that I can stay informed
US2	As a user, I want to receive alerts for extreme weather conditions
US3	As a user, I want to see risk levels (Low/Medium/High) for my location
US4	As an admin, I want to manage weather and alert data
US5	As a user, I want emergency preparedness guidelines during disasters
US6	As a system, I want to analyze weather data to generate risk indicators

Thankyou